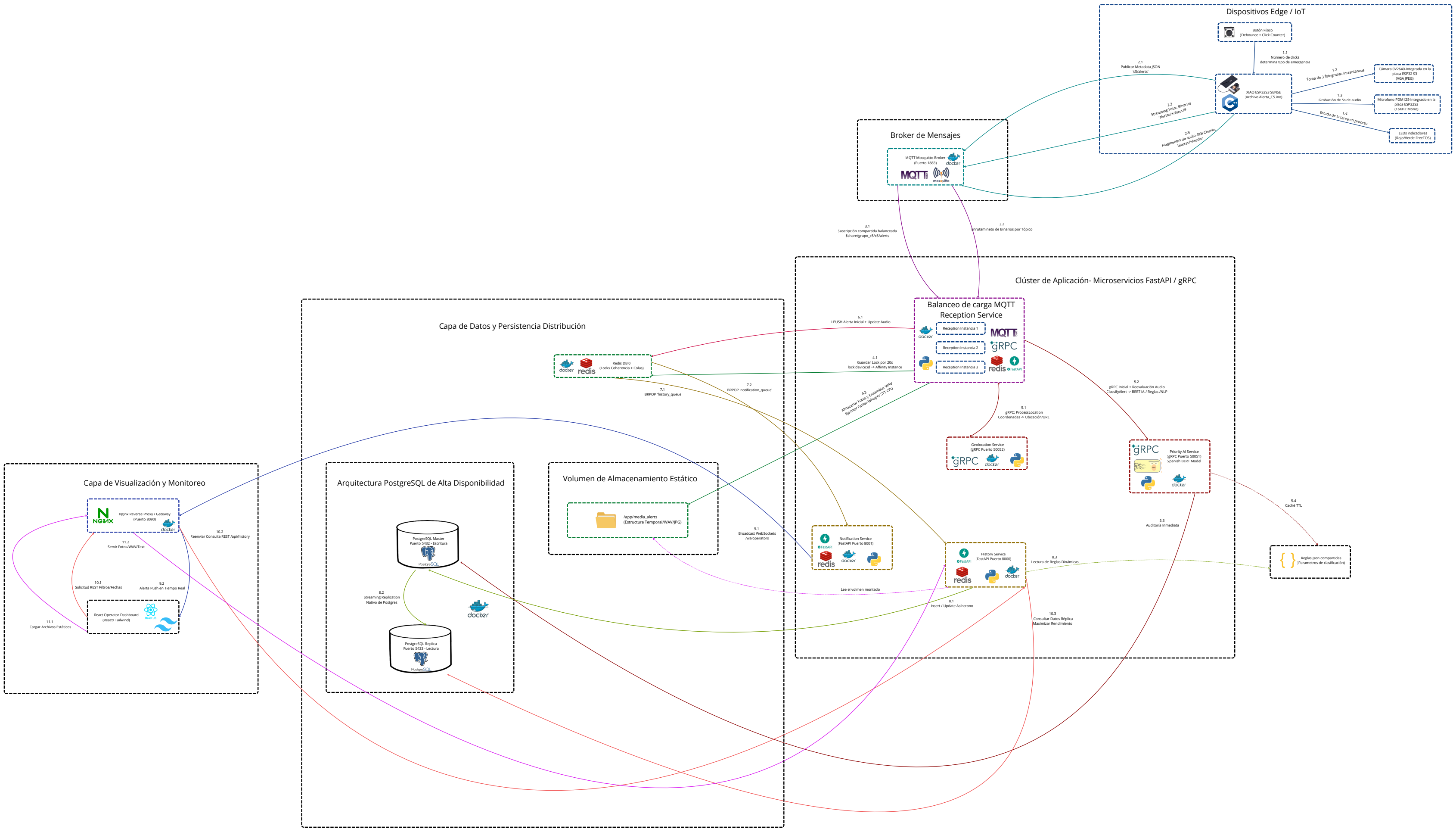




Explicación Paso a Paso del Flujo de Datos

ETAPA A: Activación y Recolección en el Dispositivo de Campo (Edge IoT)

Paso 1.1 (Detección Inteligente del Botón): El usuario presiona el botón físico del dispositivo XIAO ESP32S3 Sense. El código Alerta_C5.ino ejecuta una lógica de filtrado de rebote (debounce). Si detecta pulsaciones adicionales dentro de una ventana de tiempo de 2 segundos (CLICK_TIMEOUT), calcula el tipo de emergencia:

- 1 click = "Urgencias Médicas"
 - 2 clicks = "Seguridad Pública"
 - 3 clicks = "Protección Civil"
 - 4 o más clicks = "Fallas de Servicios y Movilidad"
-
- **Pasos 1.2, 1.3 y 1.4 (Captura en Paralelo):** Al expirar el tiempo de espera, el sistema cambia al estado CAPTURING. Utiliza FreeRTOS para inicializar una tarea en paralelo (blinkTask) montada en el Core 1 encargada de destellar el LED Rojo como indicador visual de pánico. Simultáneamente, el Core 0 realiza la captura de pruebas de entorno de forma directa en la memoria PSRAM:
 - Descarta un frame viejo de la cámara OV2640 para limpiar el buffer de iluminación y toma 3 fotografías consecutivas en formato VGA/JPEG.
 - Registra 5 segundos de audio ambiental continuo utilizando el micrófono digital PDM mediante la interfaz de alta velocidad I2S.

ETAPA B: Ingestión Concurrente por MQTT y Mecanismo de Afinidad (Broker e Ingesta)

- **Pasos 2.1, 2.2 y 2.3 (Transmisión por Fragmentación):** El dispositivo se conecta a la red inalámbrica local configurada y transmite los datos en ráfagas concurrentes hacia el Broker MQTT Mosquitto mediante tópicos especializados:
 - **c5/alerts:** Publica los metadatos iniciales estructurados en formato JSON (ID de dispositivo, coordenadas fijas de latitud/longitud, tipo de emergencia detectado y marca de tiempo UNIX).
 - **alertas/XIAO_SENSE_01/fotos/#:** Publica los buffers de las 3 imágenes.
 - **alertas/XIAO_SENSE_01/audio:** Publica el audio binario en bloques eficientes de 4KB (Chunking) debido a las limitaciones de memoria de los paquetes de red, enviando la palabra clave "FIN" al concluir.
- **Pasos 3.1 y 3.2 (Balanceo de Carga por Suscripción Compartida):** Para soportar alta concurrencia en producción, los microservicios de recepción se escalan horizontalmente en 3 instancias independientes (reception1, reception2, reception3). Utilizan la característica de Suscripciones Compartidas de MQTT v5.0 (\$share/grupo_c5/c5/alerts). Mosquitto distribuye el mensaje JSON inicial a un solo nodo del cluster de forma balanceada (por ejemplo, reception1).
- **Paso 4.1 (Mecanismo de Lock y Afinidad en Redis):** Dado que los fragmentos de fotos y audio llegarán milisegundos después del JSON de forma asíncrona, la instancia ganadora (reception1) escribe inmediatamente una llave en Redis (lock:device:XIAO_SENSE_01) con una expiración estricta de 20 segundos. Esto le dice al resto del cluster: "Yo gané el procesamiento de este evento, rediríjanme los binarios subsecuentes de este dispositivo".

- **Paso 4.2 (Ensamblado y Transcripción por Inteligencia Artificial Local):** reception1 crea una estructura de directorios legibles en el volumen compartido montado en disco (./media_alerts/{device_id}_{timestamp}). Conforme recibe los fragmentos de audio por MQTT, los concatena en un archivo .pcm temporal. Al leer la bandera "FIN", el microservicio convierte los bytes crudos en un archivo de audio estructurado estándar .wav (Mono, 16-bit, 16000Hz). Inmediatamente, ejecuta el modelo de procesamiento de lenguaje natural Faster-Whisper (base optimizado con cuantización int8) directamente en CPU para realizar el Speech-to-Text, transcribiendo el audio de la víctima en un archivo de texto plano ejecutable {device_id}_transcripcion.txt.

ETAPA C: Enriquecimiento de Datos, gRPC y Evaluación de IA (Capa de Negocio)

- **Paso 5.1 (Geolocalización Síncrona por gRPC):** El microservicio de recepción realiza una llamada RPC síncrona y de ultra baja latencia al servicio interno geolocation a través del puerto 50052. Este servicio toma la latitud y longitud, calcula la zona de proximidad basada en sectores específicos del municipio de Jilotepec y retorna un enlace dinámico formateado para Google Maps.
- **Paso 5.2 (Clasificación de Prioridad y Despacho con Spanish BERT):** reception1 se comunica por gRPC (puerto 50051) con el microservicio priority. Este servicio procesa la emergencia en dos fases:
 - **Fase Inicial:** Clasifica la alerta usando únicamente el tipo de click físico enviado por el botón del ESP32.
 - **Fase de Reevaluación:** Al terminar la transcripción del audio, se envía el texto recuperado por Whisper. El motor corre un pipeline de clasificación de texto (Zero-Shot Classification) apoyado en el modelo de lenguaje avanzado BERT en Español (Recognai/bert-base-

spanish-wwm-cased-xnli). El servicio lee dinámicamente un conjunto de reglas de negocio en reglas.json respaldado por una caché optimizada con tiempo de vida (TTL) de 10 segundos.

- **Pasos 5.3 y 5.4 (Guardrailes y Auditoría):** El motor aplica lógica de toma de decisiones jerárquica:
 - **Guardrail Crítico:** Si el audio transcrito contiene palabras clave críticas explícitas, el motor sobrescribe la salida de la IA y eleva la alerta a nivel "CRITICO", asignando unidades tácticas avanzadas.
 - **Filtro de Descarte:** Si el nivel de confianza de la IA determina que el mensaje es una broma o reporte falso, se degrada a nivel "MEDIO" y se asigna una unidad de "Solo Monitoreo".
 - Toda ejecución es registrada síncronamente en la tabla de base de datos priority_audit de postgres-master para auditoría técnica de los tiempos de inferencia en milisegundos (execution_time_ms) y puntajes de confianza (confidence_score).

ETAPA D: Encolamiento Asíncrono, Persistencia y Notificación en Tiempo Real

- **Paso 6.1 (Arquitectura Dirigida por Eventos y Desacoplamiento):** Una vez enriquecida la información, el servicio de recepción empaqueta los datos finales y realiza una operación de inserción en cola LPUSH en Redis, enviando el JSON tanto a la cola history_queue como a notification_queue de forma independiente para que los hilos de persistencia y UI no interfieran entre sí.
- **Pasos 7.1 y 8.1 (Persistencia Asíncrona en PostgreSQL Maestro):** El microservicio history cuenta con un hilo de ejecución en segundo plano (Worker Thread) que realiza una lectura bloqueante BRPOP de la cola history_queue. Si el mensaje corresponde a una alerta nueva, realiza un

mapeo relacional mediante SQLAlchemy e inserta el registro en la base de datos postgres-master. Si recibe un payload con la acción "UPDATE_PRIORITY" (derivado de la reevaluación del audio), busca el registro existente y actualiza las columnas de prioridad, tipo de emergencia y unidad de respuesta asignada en caliente.

- **Paso 8.2 (Replicación del Motor de Datos):** El nodo de base de datos postgres-master replica toda transacción de forma asíncrona mediante el protocolo nativo de replicación por transmisión de registros (Streaming Replication) hacia el nodo secundario de solo lectura postgres-replica (mapeado en el puerto externo 5433).
- **Pasos 7.2, 9.1 y 9.2 (Notificación Push en Tiempo Real por WebSockets):** El microservicio notification actúa como un consumidor asíncrono de notification_queue. Si existen operadores de despacho en línea, toma la alerta y realiza un broadcast enviando los datos mediante conexiones persistentes bi-direccionales de WebSockets (/ws/operators), las cuales cruzan de forma segura a través del proxy inverso NGINX Gateway en el puerto 8090 hasta renderizarse instantáneamente en la interfaz de usuario desarrollada en React + Tailwind CSS.
 - **Mecanismo de Tolerancia a Fallos:** Si no hay operadores en línea o la entrega falla, el servicio vuelve a insertar la alerta al inicio de la cola en Redis para asegurar cero pérdida de datos.
- **Pasos 10.1 a 11.2 (Consultas de Historial y Consumo de Archivos Multimedia):** Cuando un operador en el panel de React requiere aplicar filtros históricos, buscar reportes por fechas o zonas geográficas, la SPA realiza peticiones HTTP REST tradicionales al endpoint /api/history. NGINX redirige esta petición al servicio history, el cual ejecuta la consulta exclusivamente sobre el nodo postgres-replica, liberando de carga

transaccional al nodo Maestro. Los elementos estáticos como las fotos JPEG, archivos de audio WAV y logs de texto se consumen mapeando directamente las rutas relativas devueltas por la base de datos que exponen los archivos almacenados en el almacenamiento compartido.